

Naan Mudhalvan Project Report

Project: Implement Client Script & UI Policy (Incident) | ServiceNow System Administration

Project Team (SkillWallet)

Shalini B (Team Lead), Shalini Kamatchi A,
Srinithi S, Sowmiya S, Varshini K S

Program Track

Information Technology | Group Project
(Progress: 90%)

1. Agile Planning Logic & Terminology

A **Sprint** is a fixed period or duration in which a team works to complete a set of tasks.

An **Epic** is a big task or project that is too large to complete in one sprint. It is broken down into smaller tasks (stories) that can be completed over multiple sprints.

A **Story** is a small task. It is part of an **Epic**.

A **Story Point** is a number that represents how much effort a story takes to complete (usually in form of Fibonacci series):

- **1** - Very Easy task
- **2** - Normal task
- **3** - Moderate task
- **5** - Difficult task

2. Sprint Breakdown & Velocity

Sprint 1

- **Data Collection (Epic 1):** Gathering Data (USN1) [2 pts], Loading Data (USN2) [1 pt]
- **Data Preparation (Epic 2):** Handling Missing Values (USN3) [3 pts], Creating Fields (USN4) [3 pts], Handling Inconsistency in Data (USN5) [3 pts]

Total Story Point in Sprint 1 = 2 + 1 + 3 + 3 + 3 = 12

Sprint 2

- **Data Visualization (Epic 3):** Bar Chart (USN6) [2 pts], Pie Chart (USN7) [2 pts], Line Chart (USN8) [2 pts], Map (USN9) [4 pts]
- **Dashboard (Epic 4):** Developing Dashboard (USN10) [5 pts]
- **Story (Epic 5):** Developing Story (USN11) [5 pts]

Total Story Point in Sprint 2 = 2 + 2 + 2 + 4 + 5 + 5 = 20

3. Velocity Calculation

- **Total Story Points:** Sprint 1 (12) + Sprint 2 (20) = **32**
- **Number of Sprints:** **2**
- **Velocity Formula:** Velocity = Total Story Points Completed / Number of Sprints
- **Calculation:** 32 / 2 = 16 (Story Points per Sprint)

Your team's velocity is 16 Story Points per Sprint.

4. Project Overview

This project focuses on implementing **Client Scripts** and **UI Policies** on the ServiceNow **Incident** form to improve user experience, enforce field behavior, and reduce incorrect data entry. The implementation dynamically controls form fields based on user actions and incident conditions without requiring the form to be submitted.

5. Milestone Implementation Details

Milestone 1: Create UI Policy on Incident

Objective: Enforce mandatory fields or visibility rules dynamically on the Incident form based on selected criteria (e.g., when State is set to 'Resolved').

- **Table:** Incident [incident]
- **UI Policy Condition:** State = Resolved (6)
- **Action:** Set 'Close code' and 'Close notes' to Mandatory and Visible.

Milestone 2: Create UI Policy Action - Urgency

Objective: Dynamically restrict or modify form fields based on the Urgency and Priority fields to streamline technician workflows.

- **Table:** Incident [incident]
- **UI Policy Condition:** Urgency = 1 - High
- **Action:** Make 'Work notes' mandatory to ensure justification is captured for high-urgency incidents. Reverse if condition is false.

Milestone 3: Create onChange Client Script

Objective: Execute client-side JavaScript when a specific field's value changes (e.g., auto-populating assignment group based on Category).

- **Table:** Incident [incident] | **UI Type:** All
- **Field Name:** category | **Type:** onChange

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) { if (isLoading || newValue === '') return; if (newValue == 'software') { g_form.setValue('assignment_group', 'Software Support'); } else if (newValue == 'hardware') { g_form.setValue('assignment_group', 'Hardware Support'); } }
```

Milestone 4: Create onSubmit Client Script (Save Validation)

Objective: Validate form data before submission. Prevent incident submission if short description is missing specific keywords or details.

```
function onSubmit() { var shortDesc = g_form.getValue('short_description'); if (shortDesc.length < 10) { alert('Short description must be at least 10 characters long.');
```

Milestone 5: Create onCellEdit Client Script

Objective: Handle list-editor cell modifications to validate changes made directly from list views.

```
function onCellEdit(sysIDs, table, oldValues, newValues, callback) { var saveAndClose = true; if (newValues == '1' && !g_user.hasRole('itil')) { alert('Only ITIL users can set priority to Critical.');
```

Milestone 6: Testing the Configuration

Objective: Verify that UI Policies and Client Scripts execute correctly in the ServiceNow instance.

- Tested Incident form load behavior and verified conditional mandatory fields.
- Validated onChange and onSubmit scripts using test user accounts.

Conclusion

The ServiceNow Incident Client Script & UI Policy project was successfully planned, structured using Agile sprints, and executed through all 6 milestones. The implemented configurations successfully automate form behavior and enhance data integrity.